

Code:

```
#include <stdio.h>

#define MAXN 200005

typedef long long ll;

ll tree[4 * MAXN]; ll lazy_a[4 * MAXN]; // base increment ll lazy_d[4 * MAXN]; // step
increment ll arr[MAXN];

void build(int k, int l, int r) { lazy_a[k] = 0; lazy_d[k] = 0; if (l == r) { tree[k] = arr[l]; return; } int
mid = (l + r) / 2; build(2 * k, l, mid); build(2 * k + 1, mid + 1, r); tree[k] = tree[2 * k] + tree[2 *
k + 1]; }

void apply_lazy(int k, int l, int r, ll a, ll d) { ll len = r - l + 1; tree[k] += len * a + d * (len * (len -
1) / 2); lazy_a[k] += a; lazy_d[k] += d; }

void push(int k, int l, int r) { if (lazy_a[k] == 0 && lazy_d[k] == 0) return; int mid = (l + r) / 2;
apply_lazy(2 * k, l, mid, lazy_a[k], lazy_d[k]); apply_lazy(2 * k + 1, mid + 1, r, lazy_a[k] +
lazy_d[k] * (mid - l + 1), lazy_d[k]); lazy_a[k] = 0; lazy_d[k] = 0; }

void update(int k, int l, int r, int ql, int qr) { if (ql <= l && r <= qr) { apply_lazy(k, l, r, l - ql + 1,
1); return; } push(k, l, r); int mid = (l + r) / 2; if (ql <= mid) update(2 * k, l, mid, ql, qr); if (qr >
mid) update(2 * k + 1, mid + 1, r, ql, qr); tree[k] = tree[2 * k] + tree[2 * k + 1]; }

ll query(int k, int l, int r, int ql, int qr) { if (ql <= l && r <= qr) return tree[k]; push(k, l, r); int
mid = (l + r) / 2; ll ans = 0; if (ql <= mid) ans += query(2 * k, l, mid, ql, qr); if (qr > mid) ans
+= query(2 * k + 1, mid + 1, r, ql, qr); return ans; }

int main() { int n, q; if (scanf("%d %d", &n, &q) != 2) return 0;

for (int i = 1; i <= n; i++) scanf("%lld", &arr[i]);

build(1, 1, n);

while (q--) {
    int type, a, b;
    scanf("%d %d %d", &type, &a, &b);
    if (type == 1) {
        update(1, 1, n, a, b);
    } else {
        printf("%lld\n", query(1, 1, n, a, b));
    }
}
```

```
}  
return 0;  
  
}
```

Input:

```
5 3  
4 2 3 1 7  
2 1 5  
1 1 5  
2 1 5
```

Output:

```
17  
  
32
```